

```

1  type symbol(index: usize, symbol: String)
2  type length(n: usize)
3
4  rel digit = {"0", "1", "2", "3", "4", "5", "6", "7", "8", "9"}
5
6  type term(value: f32, begin: usize, end: usize)
7  rel term(x as f32, b, b + 1) = symbol(b, x) and digit(x)
8
9  type mult_div(value: f32, begin: usize, end: usize)
10 rel mult_div(x, b, r) = term(x, b, r)
11 rel mult_div(x * y, b, e) = mult_div(x, b, m) and symbol(m, "*") and
12                             term(y, m + 1, e)
13 rel mult_div(x / y, b, e) = mult_div(x, b, m) and symbol(m, "/") and
14                             term(y, m + 1, e)
15
16 type add_minus(value: f32, begin: usize, end: usize)
17 rel add_minus(x, b, r) = mult_div(x, b, r)
18 rel add_minus(x + y, b, e) = add_minus(x, b, m) and symbol(m, "+") and
19                             mult_div(y, m + 1, e)
20 rel add_minus(x - y, b, e) = add_minus(x, b, m) and symbol(m, "-") and
21                             mult_div(y, m + 1, e)
22
23 type result(value: f32)
24 rel result(y) = add_minus(y, 0, 1) and length(1)
25
26 query result

```

Fig. 26. Scallop code for HWF.

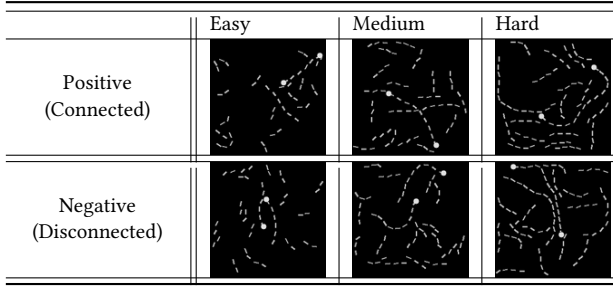


Fig. 27. Examples of positive and negative for each difficulty level.

C.3 Pathfinder

Problem Definition. Pathfinder is a task that initially set out to test the long-range reasoning ability of neural networks. It is adopted into a benchmark called Long Range Arena (LRA) [Tay et al. 2020] to evaluate the long-range reasoning ability of neural networks, or more specifically the Transformers [Vaswani et al. 2017]. In this task, we are provided with images with dark backgrounds and two white dots possibly connected by dashes. The model is then required to make a binary prediction on whether the two dots are connected or not. There are multiple versions of the dataset: one consisting of only 32×32 images (Path) and the other one with 128×128 (Path-X). Both versions

```

1 // Input from neural networks
2 type dash(i8, i8)
3 type dot(i8)
4
5 // Connectivity check
6 rel path(x, y) = dash(x, y) or path(x, z) and dash(z, y)
7 rel connected() = dot(x), dot(y), path(x, y), x != y

```

Fig. 28. Scallop code for Pathfinder.

have 3 difficulties, easy, normal, and hard, depending on the length of the dashes connecting the two dots – longer means harder. Table 27 shows the 3 difficulties each with one positive and one negative sample from Path-X.

Architecture. Going to the extreme of symbolism one would use the individual symbol to represent each small dash and dot. However, doing so would mean that the perception network, presumably a CNN, needs to segment the image precisely to extract all dashes and dots as entities. At the same time, the reasoning module bears an extra burden when processing individual connectivities. Instead, we employ a simple grid-based connectivity graph: each node represents a conceptual “dot” and each edge represents a conceptual “dash”. We ask neural networks to take in the image and output a feature vector with the size of the number of dots plus the number of edges. Each logit in this vector is then treated as a probability of that “dot” or “dash”. The Scallop program we use for this task, as shown in Fig. 28, is simply a transitive closure that checks whether there are two distinct dots connected by dashes.

Experimental Setup. There are 600K training data points for both PATH and PATH-X, which are images of 32×32 and 128×128 pixels respectively, and the y-label represents whether there exists a path that connects the two dots on the image. We recognize the “dash” and “dots” relations through a 4-layer CNN, and the local connectivity is recognized through a 2-layer MLP classifier. We set the learning rate to 0.0001, the batch size to 64, and the number of epochs to 100.

C.4 PacMan-Maze

PacMan-Maze is a reinforcement learning based planning application, which aims to lead the actor to the flag without running into any enemy. We show the Scallop code for this task in Fig. 29. In addition to the code shown in Section 2, we added additional “integrity constraint violation” terms that need to be minimized during learning.

Experimental Setup. We set the size of the maze to be a 5×5 matrix, and the maximum number of enemies that exists in the arena to be 5. In the training process, we set the batch size to be 24, the memory replay buffer size to be 3000, and the maximum number of actions in an episode to be 30. We update the target net every 10 epochs, we set the exploration rate to be 0.9, its falloff to be 0.98, and the learning rate is set to 0.0001.

C.5 CLUTRR

With CLUTRR [Sinha et al. 2019] we step into the domain of NLP on kinship reasoning. Each data point in the CLUTRR dataset consists of a natural language passage, a query, and its answer. The passage tells a story about a family’s daily activities; the query is a tuple of two names, denoting the target pair of persons that we want to predict the relation for; the answer is among the 20 basic relations such as “mother”, “uncle”, and “daughter-in-law”. The dataset is further categorized into

```

1  // Input from neural networks
2  type grid_node(x: usize, y: usize) // 0.99 for each grid cell
3  type actor(x: usize, y: usize)
4  type goal(x: usize, y: usize)
5  type enemy(x: usize, y: usize)
6
7  // Basic connectivity
8  const UP = 0, RIGHT = 1, DOWN = 2, LEFT = 3
9  rel safe_node(x, y) = grid_node(x, y), not enemy(x, y)
10 rel edge(x, y, x, yp, UP) = safe_node(x, y), safe_node(x, yp), yp == y + 1
11 rel edge(x, y, xp, y, RIGHT) = safe_node(x, y), safe_node(xp, y), xp == x + 1
12 rel edge(x, y, x, yp, DOWN) = safe_node(x, y), safe_node(x, yp), yp == y - 1
13 rel edge(x, y, xp, y, LEFT) = safe_node(x, y), safe_node(xp, y), xp == x - 1
14
15 // Get the next position
16 rel next_pos(xp, yp, a) = actor(x, y), edge(x, y, xp, yp, a)
17
18 // Path for connectivity; will condition on no enemy on the path
19 rel path(x, y, x, y) = next_pos(x, y, _)
20 rel path(x1, y1, x3, y3) = path(x1, y1, x2, y2), edge(x2, y2, x3, y3, _)
21
22 // Get the next action
23 rel next_action(a) = next_pos(x, y, a), goal(gx, gy), path(x, y, gx, gy)
24
25 // Constraint violation
26 type too_many_goal(), too_many_actor(), too_many_enemy()
27 rel too_many_goal() = n := count(x, y: goal(x, y)), n > 1
28 rel too_many_actor() = n := count(x, y: actor(x, y)), n > 1
29 rel too_many_enemy() = n := count(x, y: enemy(x, y)), n > 5
30 rel violation() = too_many_goal() or too_many_enemy() or too_many_actor()

```

Fig. 29. Scallop code for PacMan-Maze.

```

1  // Define kinships belong to the Relation type
2  type Relation = usize
3
4  // Input from neural networks
5  type question(sub: String, obj: String)
6  type kinship(rela: Relation, sub: String, obj: String)
7  type composition(Relation, Relation, Relation)
8
9  // Rules to derive the final answer
10 rel kinship(r3, x, z) = composition(r1, r2, r3), kinship(r1, x, y),
11                          kinship(r2, y, z), x != z
12 rel answer(r) = question(s, o), kinship(r, s, o)

```

Fig. 30. Scallop code for CLUTRR.